Heap Sort

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercício 13; partes dos Exercícios 2, 3 e 14).

Propriedade	Valor
Estratégia	constrói um **heap máximo** e extrai repetidamente o maior elemento para o fim
Construção do heap	$\Theta(n)$
Melhor / médio / pior caso	$\Theta(n \log n)$ nos três
Espaço auxiliar	$\Theta(1)$ com `heapify` iterativo; $O(\log n)$ de pilha com `heapify` recursivo
In-place	Sim
Estável	Não

---

1. O que é um heap

Um **heap máximo** é uma **árvore binária completa** em que **cada pai é maior ou igual aos seus filhos**. O maior elemento fica sempre na **raiz**.

A árvore é guardada no próprio vetor, nível por nível:

Relação	Índice (vetor começando em 0)
filho esquerdo de `i`	`2*i + 1`
filho direito de `i`	`2*i + 2`
pai de `i`	`(i - 1) // 2`
folhas	posições `n//2` até `n-1`

Exemplo: o vetor `[10, 5, 3, 4, 1]` representa

```
        10          <- pos 0 (raiz: o maior)
      /    \
     5      3       <- pos 1, 2
    / \
   4   1           <- pos 3, 4
```

O heap **não precisa estar totalmente ordenado**: basta respeitar a relação pai ≥ filhos (aqui o 4 está num nível mais baixo que o 3, embora seja maior — sem problema, pois estão em ramos diferentes).

Altura de um heap com n elementos: $\lfloor \log_2 n \rfloor$.

---

2. Ideia do algoritmo

1. **Constrói** um heap máximo com todo o vetor.
2. **Troca** a raiz (o maior) com o último elemento da região do heap.
3. **Reduz** a região do heap em 1 (o maior agora está na posição final).
4. **Restaura** a propriedade de heap a partir da raiz (`heapify`).
5. **Repete** até a região do heap ter 1 elemento.

A cada etapa, o maior elemento restante vai para a sua posição final — é um "Selection Sort" em que achar o maior custa $O(\log n)$ em vez de $O(n)$.

---

3. Implementação

```
def heapify(v, n, i):                       # "afunda" v[i] dentro do heap v[0..n-1]
    maior = i
    esquerda = 2 * i + 1
    direita = 2 * i + 2

    if esquerda < n and v[esquerda] > v[maior]:
        maior = esquerda
    if direita < n and v[direita] > v[maior]:
        maior = direita

    if maior != i:
        v[i], v[maior] = v[maior], v[i]
        heapify(v, n, maior)                # continua descendo pelo ramo trocado


def heap_sort(v):
    n = len(v)

    # Construção do heap máximo: das folhas para a raiz
    for i in range(n // 2 - 1, -1, -1):
        heapify(v, n, i)

    # Extração sucessiva do maior elemento
    for i in range(n - 1, 0, -1):
        v[0], v[i] = v[i], v[0]             # raiz (maior) vai para a posição final
        heapify(v, i, 0)                    # restaura o heap em v[0..i-1]

    return v
```

- A construção começa em `n//2 - 1` porque as posições `n//2..n-1` são **folhas**, e uma folha sozinha já é um heap.

- Processar de baixo para cima garante que, ao chamar `heapify(i)`, as subárvores dos filhos de `i` já são heaps.

`heapify` iterativo (espaço auxiliar $O(1)$)

```
def heapify_iterativo(v, n, i):
    while True:
        maior = i
        esquerda, direita = 2 * i + 1, 2 * i + 2
        if esquerda < n and v[esquerda] > v[maior]:
            maior = esquerda
        if direita < n and v[direita] > v[maior]:
            maior = direita
        if maior == i:
            return
        v[i], v[maior] = v[maior], v[i]
        i = maior
```

> **Observação importante:** o código da aula usa `heapify` **recursivo**. Embora o Heap Sort seja in-place quanto ao vetor, essa versão usa $O(\log n)$ de **pilha** durante a restauração. Para espaço auxiliar total $O(1)$, implemente o `heapify` iterativamente (acima).

---

4. Passo a passo em `[4, 10, 3, 5, 1]`

1. Construindo o heap máximo ($n = 5$, começa em `i = 5//2 - 1 = 1`):

Chamada	Ação	Vetor
início		`[4, 10, 3, 5, 1]`
`heapify(i=1)`	10 ≥ filhos 5 e 1 → nada	`[4, 10, 3, 5, 1]`
`heapify(i=0)`	4 < 10 → troca 4 ↔ 10; desce: 4 < 5 → troca 4 ↔ 5	`[10, 5, 3, 4, 1]`

Heap máximo: `[10, 5, 3, 4, 1]` (árvore da seção 1).

2. Colocando o maior na posição final, repetidamente (região ordenada após a barra):

Passo	Troca raiz ↔ último	Após a troca	Restauração (`heapify`)	Resultado
1	10 ↔ 1	`[1, 5, 3, 4 | 10]`	1 ↔ 5, depois 1 ↔ 4	`[5, 4, 3, 1 | 10]`
2	5 ↔ 1	`[1, 4, 3 | 5, 10]`	1 ↔ 4	`[4, 1, 3 | 5, 10]`
3	4 ↔ 3	`[3, 1 | 4, 5, 10]`	nada (3 ≥ 1)	`[3, 1 | 4, 5, 10]`
4	3 ↔ 1	`[1 | 3, 4, 5, 10]`	—	`[1, 3, 4, 5, 10]`

Resultado final: `[1, 3, 4, 5, 10]`.

---

5. Análise de complexidade passo a passo

5.1 Custo de um `heapify`

A cada chamada, o elemento desce **um nível** e faz trabalho constante (2 comparações + 1 troca). Para um nó de altura h:

$$T(h) = T(h-1) + c \;\Rightarrow\; T(h) = \Theta(h) \leq c \log_2 n$$

(é a recursão linear da aula de recursividade, com a altura no lugar de n). Portanto `heapify` custa $O(\log n)$, e a profundidade da pilha também é $O(\log n)$.

5.2 Construção do heap: $\Theta(n)$ (e não $n \log n$)

Conta ingênua: $n/2$ chamadas × $O(\log n)$ = $O(n \log n)$ — correta, mas **não justa**. A maioria dos nós está perto das folhas, onde `heapify` é barato:

Altura h do nó	Nº de nós (no máximo)	Custo de cada `heapify`
0 (folhas)	$\approx n/2$	0
1	$\approx n/4$	c
2	$\approx n/8$	$2c$
h	$\approx n/2^{h+1}$	hc

$$\text{Custo} \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}}\, c\, h = \frac{cn}{2} \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{h}{2^h} \leq \frac{cn}{2} \cdot 2 = cn$$

usando $\sum_{h \geq 0} h/2^h = 2$ (apêndice de 01 — Recursividade). Como a construção visita pelo menos $n/2$ nós, também é $\Omega(n)$. Logo: **construção** = $\Theta(n)$.

5.3 Fase de extrações: $\Theta(n \log n)$

São $n - 1$ extrações; na extração com heap de tamanho i, o `heapify` da raiz custa $O(\log i)$:

$$\sum_{i=2}^{n} c \log_2 i = c \log_2 (n!) = \Theta(n \log n)$$

(pois $\frac{n}{2} \log \frac{n}{2} \leq \log(n!) \leq n \log n$ — apêndice de 01). O elemento que sobe para a raiz veio do fundo do heap, então tende a descer quase até o fundo de novo: o custo $\log i$ é atingido na prática.

5.4 Total

Etapa	Complexidade
Construção do heap máximo	$\Theta(n)$
Restauração após cada extração	$O(\log n)$
Extrações sucessivas ($n-1$)	$\Theta(n \log n)$ no total
Tempo total	$\Theta(n) + \Theta(n \log n) = \boxed{\Theta(n \log n)}$
Espaço auxiliar	$O(\log n)$ de pilha (heapify recursivo) ou $\Theta(1)$ (iterativo)

Melhor, médio e pior caso: $\Theta(n \log n)$ (para chaves distintas; o algoritmo não tira proveito de entrada já ordenada).

> **Pergunta da aula:** por que construir o heap custa $\Theta(n)$, mas o algoritmo completo custa $\Theta(n \log n)$? **Porque a construção inicial é linear, mas depois são feitas $\approx n$ extrações, cada uma seguida de uma restauração que custa até $O(\log n)$.**

5.5 Por que não é estável?

A troca raiz ↔ último salta por cima de elementos iguais. Ex.: `[2a, 2b, 1]` já é heap; extrai 2a (troca com 1) → `[1, 2b | 2a]`; restaura → `[2b, 1 | 2a]`; extrai 2b → `[1, 2b, 2a]`. O 2_b terminou **antes** do 2_a: ordem relativa invertida.

---

6. Exercícios resolvidos

Exercício 13 — `v = [36, 15, 42, 8, 27, 4, 31, 19, 11, 23]` ($n = 10$)

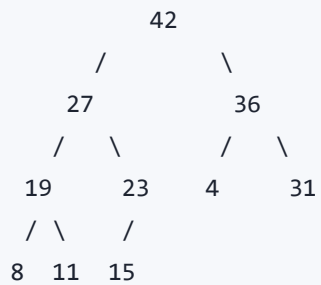
a) Vetor como árvore binária completa:

```
              36                  pos 0
          /       \
        15          42            pos 1, 2
       /   \       /   \
      8     27    4     31        pos 3, 4, 5, 6
     / \    /
   19  11  23                     pos 7, 8, 9
```

b) Construção do heap máximo (de `i = 10//2 - 1 = 4` até 0):

Chamada	Comparação	Trocas	Vetor
`heapify(4)`	27 vs filho 23	nenhuma	`[36, 15, 42, 8, 27, 4, 31, 19, 11, 23]`
`heapify(3)`	8 vs filhos 19, 11	8 ↔ 19	`[36, 15, 42, 19, 27, 4, 31, 8, 11, 23]`
`heapify(2)`	42 vs filhos 4, 31	nenhuma	`[36, 15, 42, 19, 27, 4, 31, 8, 11, 23]`
`heapify(1)`	15 vs 19, 27 → 27; depois 15 vs 23	15 ↔ 27, depois 15 ↔ 23	`[36, 27, 42, 19, 23, 4, 31, 8, 11, 15]`
`heapify(0)`	36 vs 27, 42 → 42; depois 36 vs 4, 31	36 ↔ 42	`[42, 27, 36, 19, 23, 4, 31, 8, 11, 15]`

c) Vetor após a construção: `[42, 27, 36, 19, 23, 4, 31, 8, 11, 15]`



d/e/f) Extrações (troca raiz ↔ último da região ativa, depois restauração; região ordenada após a barra):

#	Troca	Vetor após a troca	Restauração	Vetor após restaurar
1	42 ↔ 15	`[15, 27, 36, 19, 23, 4, 31, 8, 11 | 42]`	15 ↔ 36, 15 ↔ 31	`[36, 27, 31, 19, 23, 4, 15, 8, 11 | 42]`
2	36 ↔ 11	`[11, 27, 31, 19, 23, 4, 15, 8 | 36, 42]`	11 ↔ 31, 11 ↔ 15	`[31, 27, 15, 19, 23, 4, 11, 8 | 36, 42]`
3	31 ↔ 8	`[8, 27, 15, 19, 23, 4, 11 | 31, 36, 42]`	8 ↔ 27, 8 ↔ 23	`[27, 23, 15, 19, 8, 4, 11 | 31, 36, 42]`
4	27 ↔ 11	`[11, 23, 15, 19, 8, 4 | 27, 31, 36, 42]`	11 ↔ 23, 11 ↔ 19	`[23, 19, 15, 11, 8, 4 | 27, 31, 36, 42]`
5	23 ↔ 4	`[4, 19, 15, 11, 8 | 23, 27, 31, 36, 42]`	4 ↔ 19, 4 ↔ 11	`[19, 11, 15, 4, 8 | 23, 27, 31, 36, 42]`
6	19 ↔ 8	`[8, 11, 15, 4 | 19, 23, 27, 31, 36, 42]`	8 ↔ 15	`[15, 11, 8, 4 | 19, 23, 27, 31, 36, 42]`
7	15 ↔ 4	`[4, 11, 8 | 15, 19, 23, 27, 31, 36, 42]`	4 ↔ 11	`[11, 4, 8 | 15, 19, 23, 27, 31, 36, 42]`
8	11 ↔ 8	`[8, 4 | 11, 15, 19, 23, 27, 31, 36, 42]`	nada (8 ≥ 4)	`[8, 4 | 11, 15, 19, 23, 27, 31, 36, 42]`
9	8 ↔ 4	`[4 | 8, 11, 15, 19, 23, 27, 31, 36, 42]`	—	`[4, 8, 11, 15, 19, 23, 27, 31, 36, 42]`

Vetor ordenado: **`[4, 8, 11, 15, 19, 23, 27, 31, 36, 42]`**.

g) A construção é linear porque a maioria dos nós está perto das folhas e o custo do `heapify` é proporcional à **altura** do nó: $\sum_h \frac{n}{2^{h+1}} h = O(n)$ (seção 5.2). Já as $n - 1$ extrações começam sempre pela **raiz**, descendo até $\log_2 i$ níveis: $\sum \log i = \log(n!) = \Theta(n \log n)$ (seção 5.3).

h) In-place: sim (só trocas dentro do vetor; $O(1)$ com `heapify` iterativo). **Estável: não** (seção 5.5).

Exercício 3 — requisitos: pior caso $\Theta(n \log n)$, pouca memória, estabilidade não obrigatória

a) **Heap Sort**. b) Melhor, médio e pior: $\Theta(n \log n)$. c) O Quick Sort clássico pode ter partições extremas → $\Theta(n^2)$ (07). d) Se a estabilidade passar a ser obrigatória: **Merge Sort** (06), com custo de $\Theta(n)$ de memória.

Exercícios 2 e 14 — onde o Heap Sort aparece

- **Ex. 2, item 3:** garantir $\Theta(n \log n)$ mesmo no pior caso → Heap Sort ou Merge Sort.
- **Ex. 14, cenário C:** memória limitada + $\Theta(n \log n)$ no pior caso + espaço auxiliar constante → **Heap Sort** (com `heapify` iterativo).

Resoluções completas em 02 — Fundamentos.

Matérias correlatas

Recursão do `heapify`: $T(h) = T(h-1) + c = \Theta(h)$, a recursão linear da seção 4.1 de 01 — Recursividade; a pilha de profundidade $O(\log n)$ é o mesmo raciocínio da seção 9.

Séries do apêndice de 01: $\sum h/2^h = 2$ (construção em $\Theta(n)$) e $\log(n!) = \Theta(n \log n)$ (extrações).

Selection Sort: o Heap Sort é a mesma ideia de "selecionar o extremo e colocá-lo no lugar", trocando a busca linear por um heap. Veja 04 — Selection Sort.

Comparação com Merge e Quick: 02 — Fundamentos.